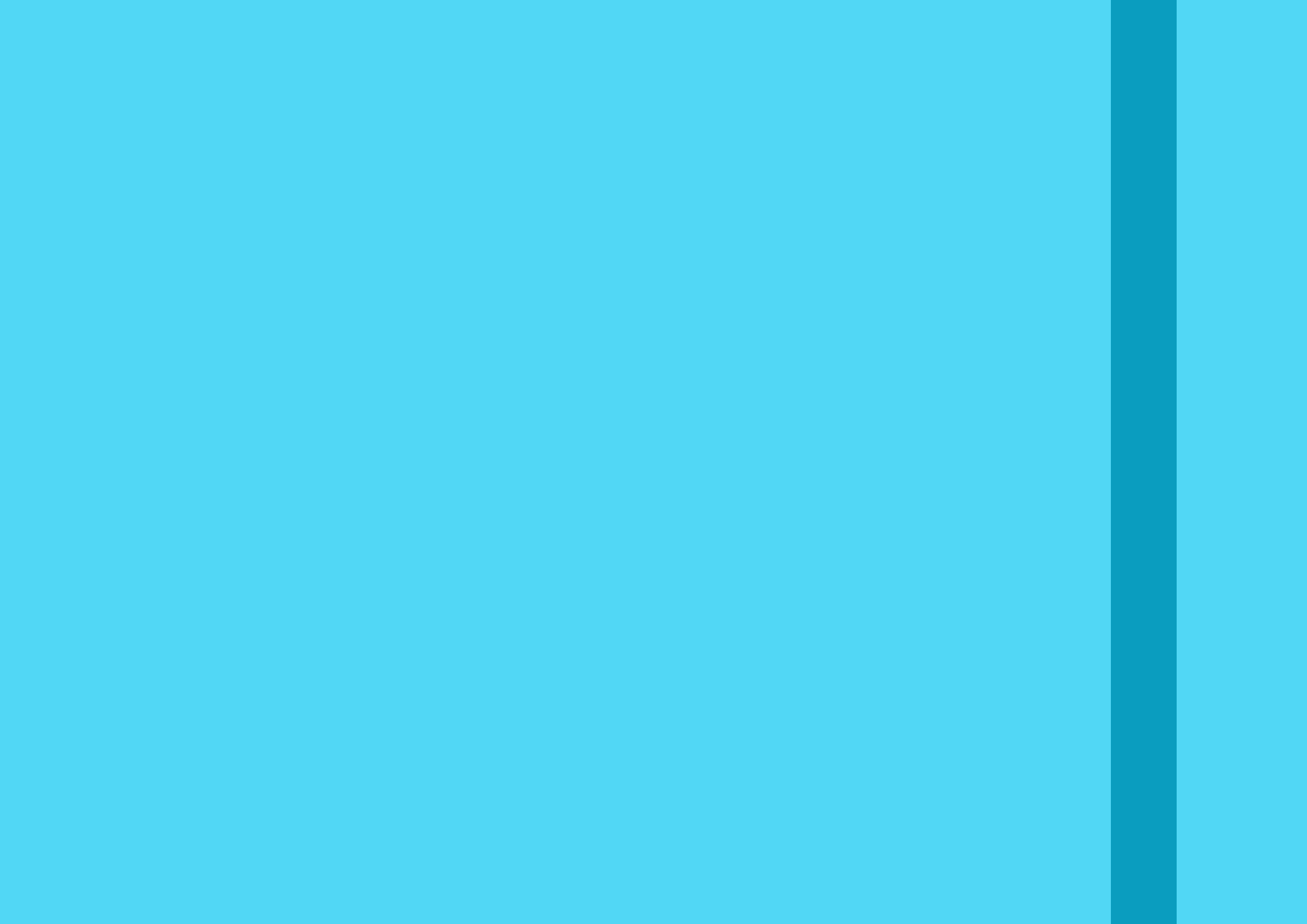




Java

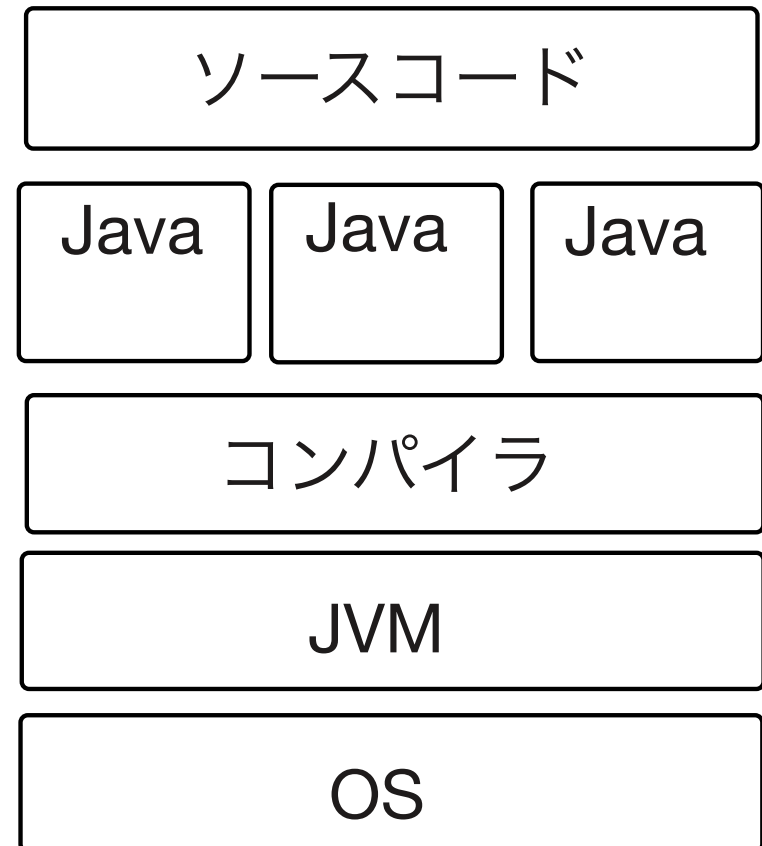
JRE: APIとJVMが内蔵

コンパイラがないため実行出来ない
(コンパイル出来ない)

JDK: APIとコンパイラ、JVMが内蔵

これだけで開発可能

Javaファイルをclassファイルへ
(ソースコードをバイトコードへ)



Spring Boot の特徴

- ・ ライブラリの一括取得 (Starter の依存関係を解決)
- ・ Auto Configuration

↳ Spring Boot が用意した Java Config を読む

① ライブラリが提供する 7つの Bean 定義

ex) Tomcat や Spring MVC, Thymeleaf などの Bean 生成

② Spring が提供する各機能の有効化

⇒ Spring Boot を使うと Java Config の Bean 定義と Annotation の 負担 を軽減することが出来る

• Auto Configuration

設定を反映させるのに必要なファイル

• application.properties or application.yml

↑ フォルダ直下に配置

・ Spring Boot の起動方法

① main メソッド内で SpringApplication.run を呼び出す
↳ application.properties を読み込む

② main メソッドを定義する際に @SpringBootApplication を付ける

@SpringBootApplication は以下 Annotation を含む

- ・ @Configuration
- ・ @EnableAutoConfiguration
- ・ @ComponentScan

• コンフィグレーション (@Configuration)

- DIのコンポーネント スキャンの範囲を定義 (@ComponentScan)

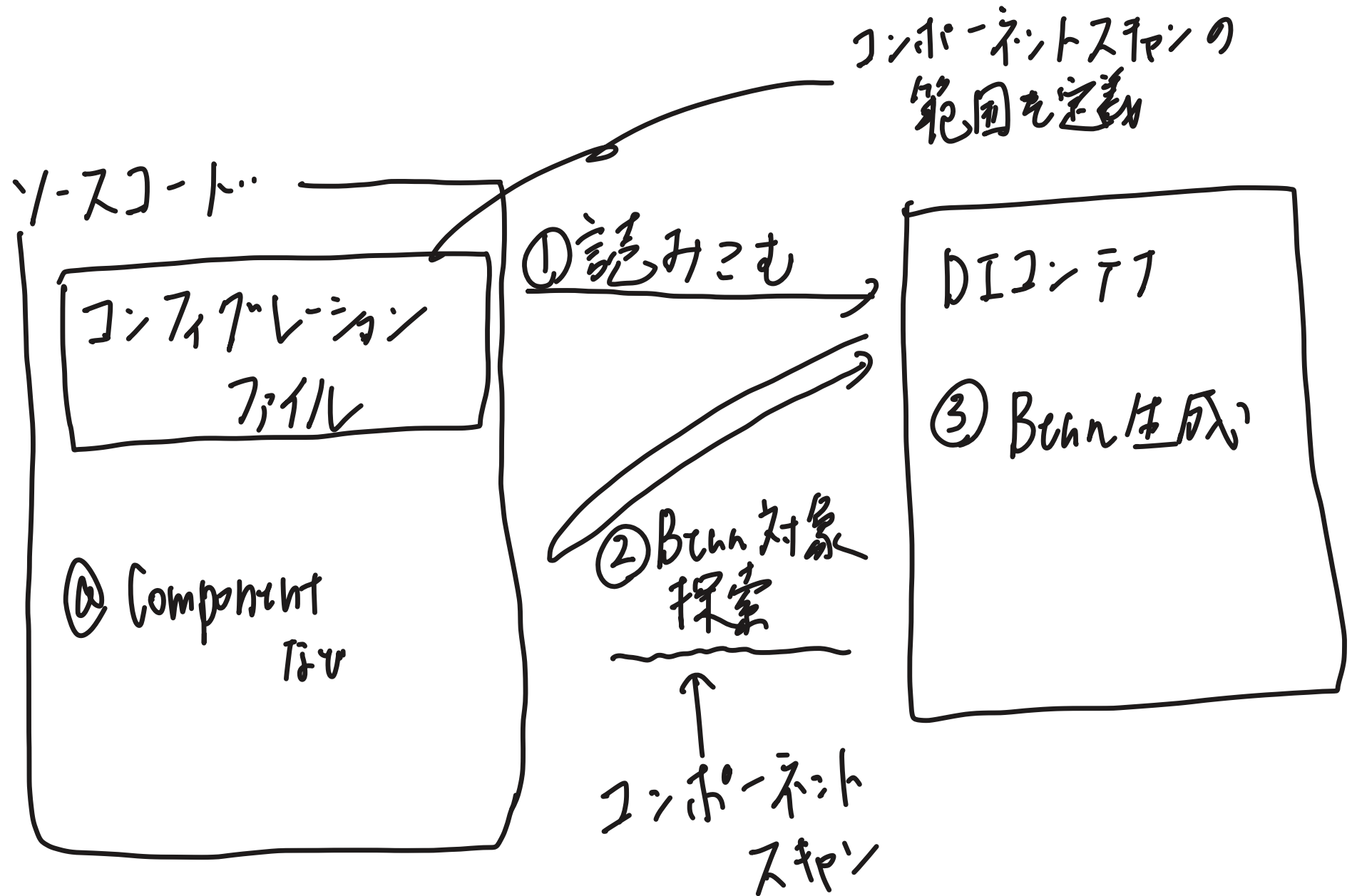
↑ コンフィグレーションファイルが属するパッケージ以下
or

特定のパッケージを指定する

- Bean定義 (Aト部ライブラリでBeanに登録が必要なものを
@Beanメソッドで登録(定義)する)

Beanに関する設定

■ DIコンテナ コンポーネントスキャン イメージ図

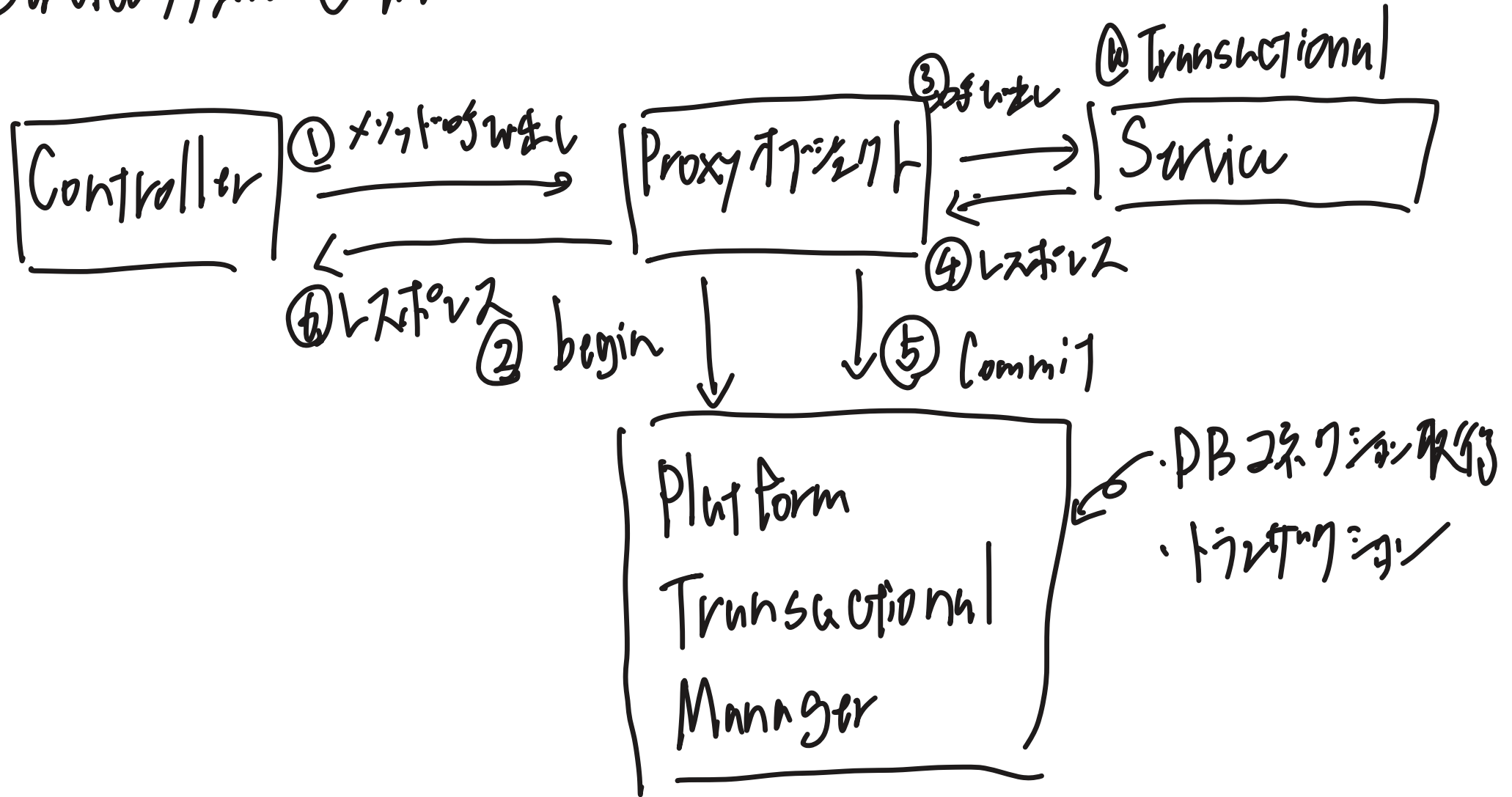


データベースアクセス方法

- ・ Spring Data JPA: O/Rマッパー 海外人気
- ・ My Batis: SQLマッパー 国内人気
- ・ JDBC Template: JDBCのマッパー 業務では使われる

• @Transactional の仕組み

Service 側で @Transactional を付けている場合



@Transactional 利用方法

- Auto Configuration を利用 ⇒ 追加対応不要

- Auto Configuration を利用しない

⇒ Configuration File (Java Config)

@EnableTransactionalManager の付加が必要

• Proxy パターン

- @Transactional or 何かと似た アノテーションを継承したアノテーション

付加されたアノテーションと同じメソッドを持つ

- Spring は Proxy パターンを Bean として扱う

Service アノテーションは Bean として扱われる

- Controller から Service アノテーションを呼び出した際

実際に呼び出されるのは Proxy パターン

- PlatformTransactionManager を呼び出して
トランザクションの取得とトランザクションを開始する

・ Proxy オブジェクトの役割

⇒ Service 712 処理する前に

Platform Transactional Manager を経由させる

⇒ Service 712 処理時に

トランザクション取得, トランザクション開始時の状態に遷移

Platform Transactional Manager の役割

① コネクションの作成

作成したコネクションは Thread Local に保存

② トランザクションの管理

Platform Transactional Manager 利用方法

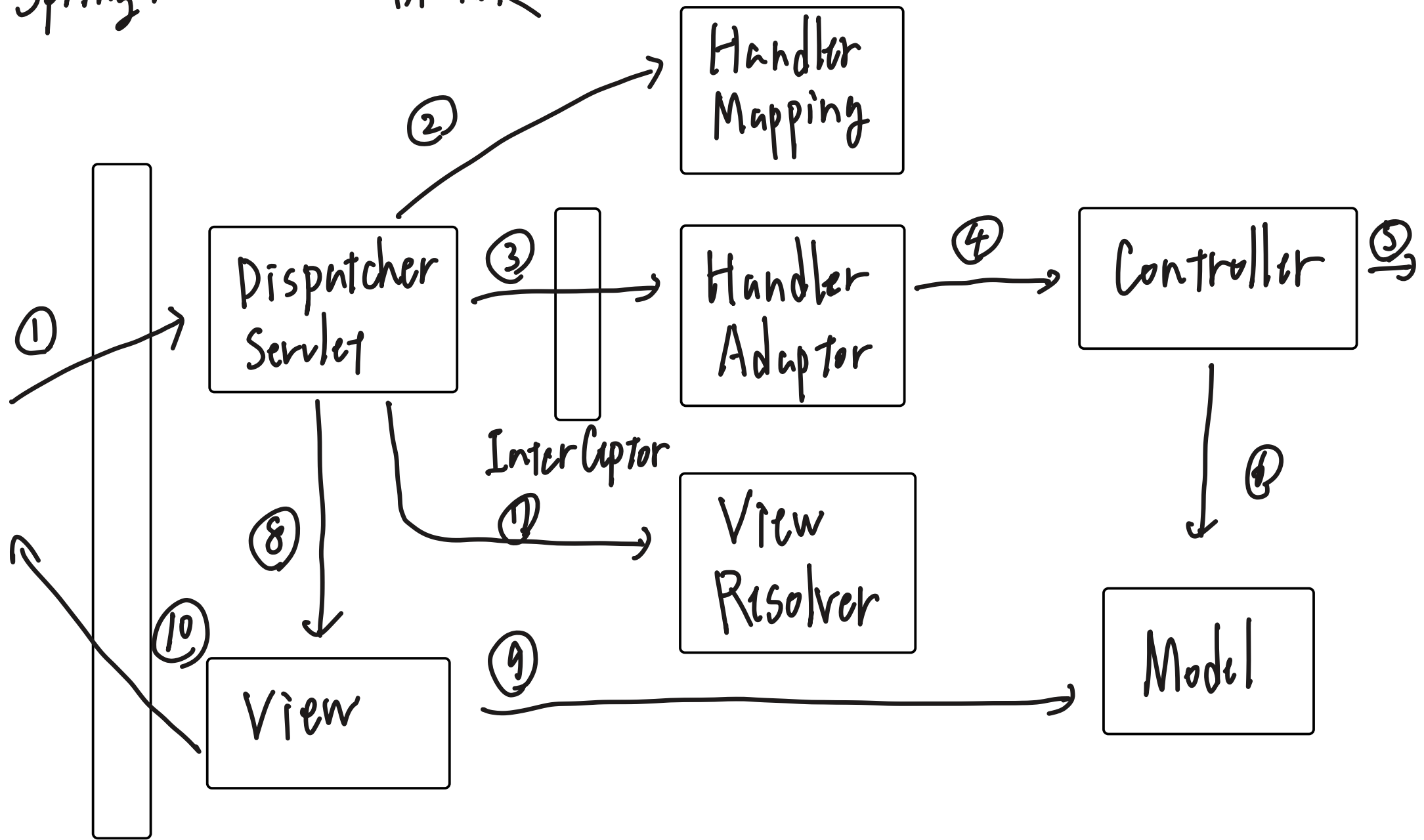
Bean 生成に

Auto Configuration を利用する $\Rightarrow$ Bean 定義不要

Auto Configuration を利用しない $\Rightarrow$ Bean 定義必要

$\hookrightarrow$ DataSource 必要

Spring MVCの全体像



Servlet Filter
(Security Filter Chainも含む)

- Dispatcher Servlet

- Spring Boot アプリケーションのフロントコントローラー

- 全てのリクエストを一括受信し適切な部品への交通整理

④ Controller を呼び出した後は Dispatcher Servlet が
前処理をした後の処理を担当している

・Xリクエスト

- URL ルーティングを行う
- リクエストの解析を行う

- Handler Mapping

リクエストされたURLからこのコントローラのメソッド(ハンドラー)を
処理するものを特定する

- ・ Handler Adapter : コントローラーの実行に引数の準備
 - ・ Handler Mapping で見つけたメソッドのハンドリングを呼び出す
 - ・ 呼び出す前に、ハンドリングの引数をリクエストデータの紐づけを行う
 - ・ @Validated がある場合、バリデーションを行う
 - ・ @RequestBody と @ResponseBody がある場合、
HttpMessage Converter (Jackson) からリクエスト・レスポンスを変換する

- ・ View Resolver : Viewを用意

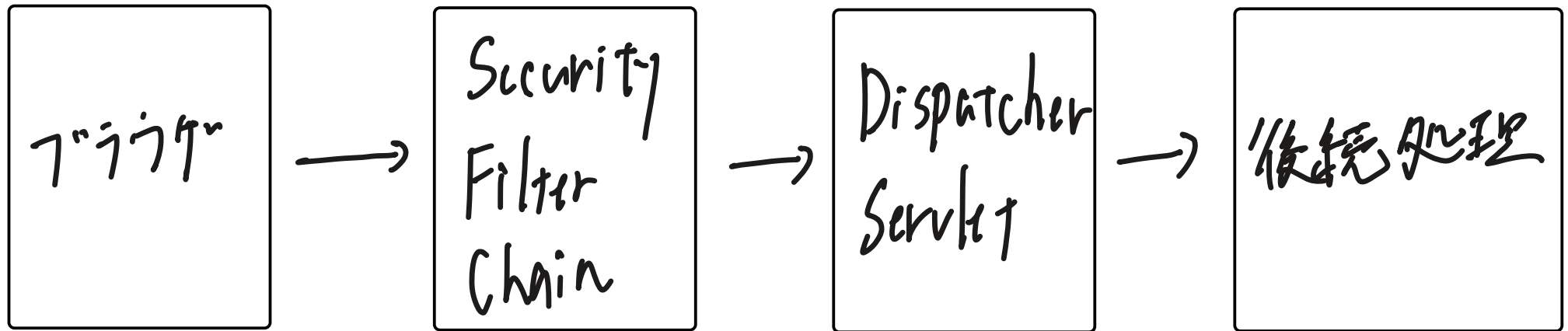
- ・ Viewを用意して Thymeleafなどのテンプレートエンジンへ
処理を引渡し

- Spring Security

代表的な認可

- ・ リクエストの認可 (アクセス可否)
- ・ メソッド認可 (呼び出し可否)
- ・ 画面表示認可 (一画面の中で表示し得る部分を制御)

Spring Security Filter Chain



↑ 認証Filter

・認可Filter

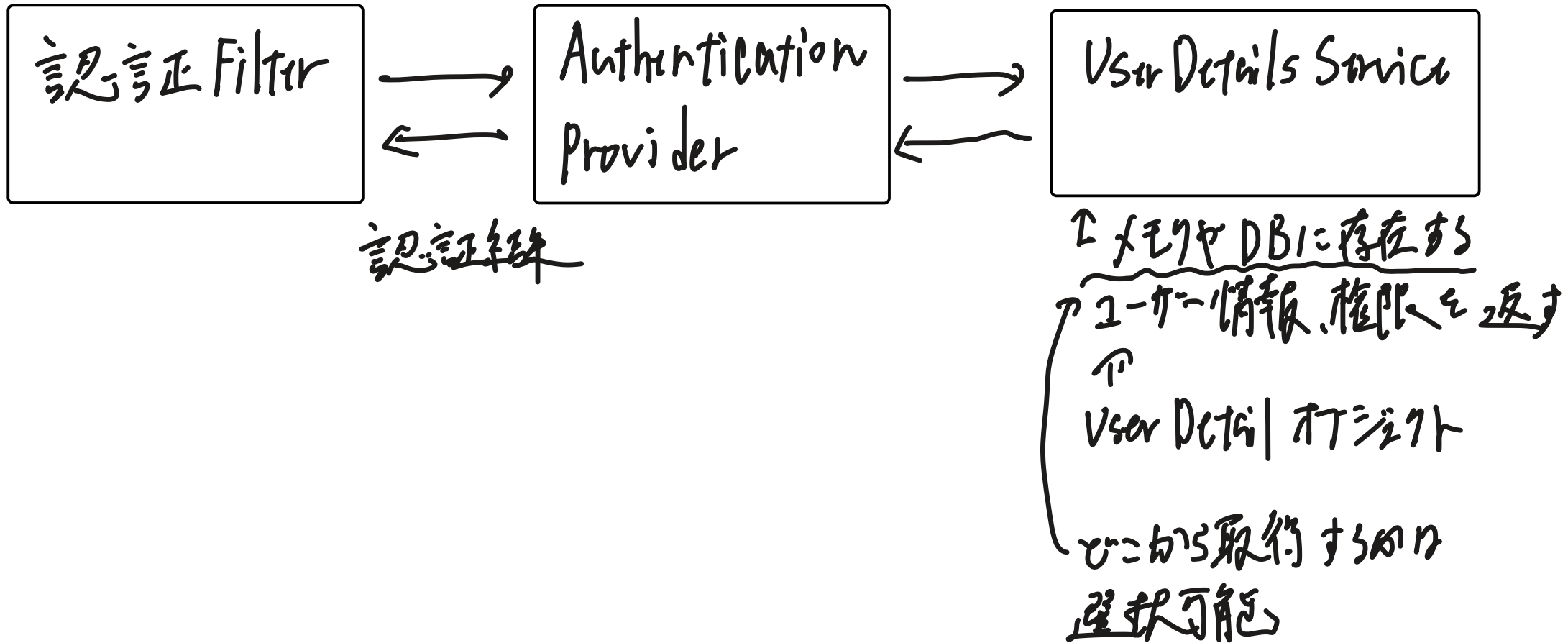
・例外ハンドリングFilter

※ Security Filter Chainの例外をハンドリングする

Security Filter Chain 利用方法

- ・ Java Config で Security Filter Chain の Bean 定義を追加し、
このような Filter を設定するのを定義する
- ・ Bean 定義方法: Java Config に `@EnableWebSecurity` を付ける

Spring Security 認証



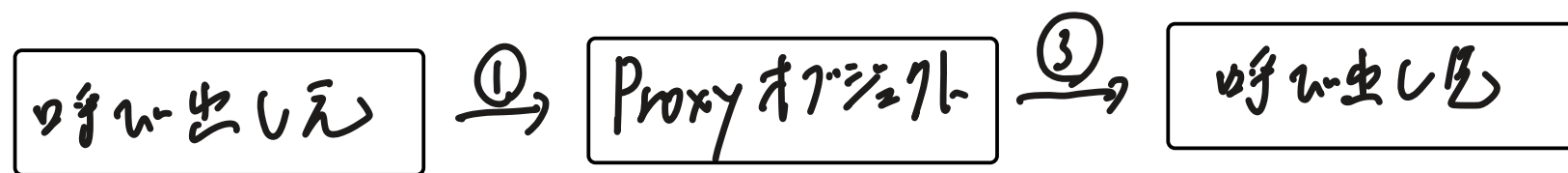
Spring Security 画面表示の認可

Thymeleaf で判定する (Spring Security の SpEL を利用)

メソッドの認可

- ・ Java Config に `@EnableMethodSecurity` を付加
- ・ 認可を行うメソッドに `@PreAuthorize (条件)` を付加

↑ 対象メソッドが呼び出される前の Proxy オブジェクトの判定

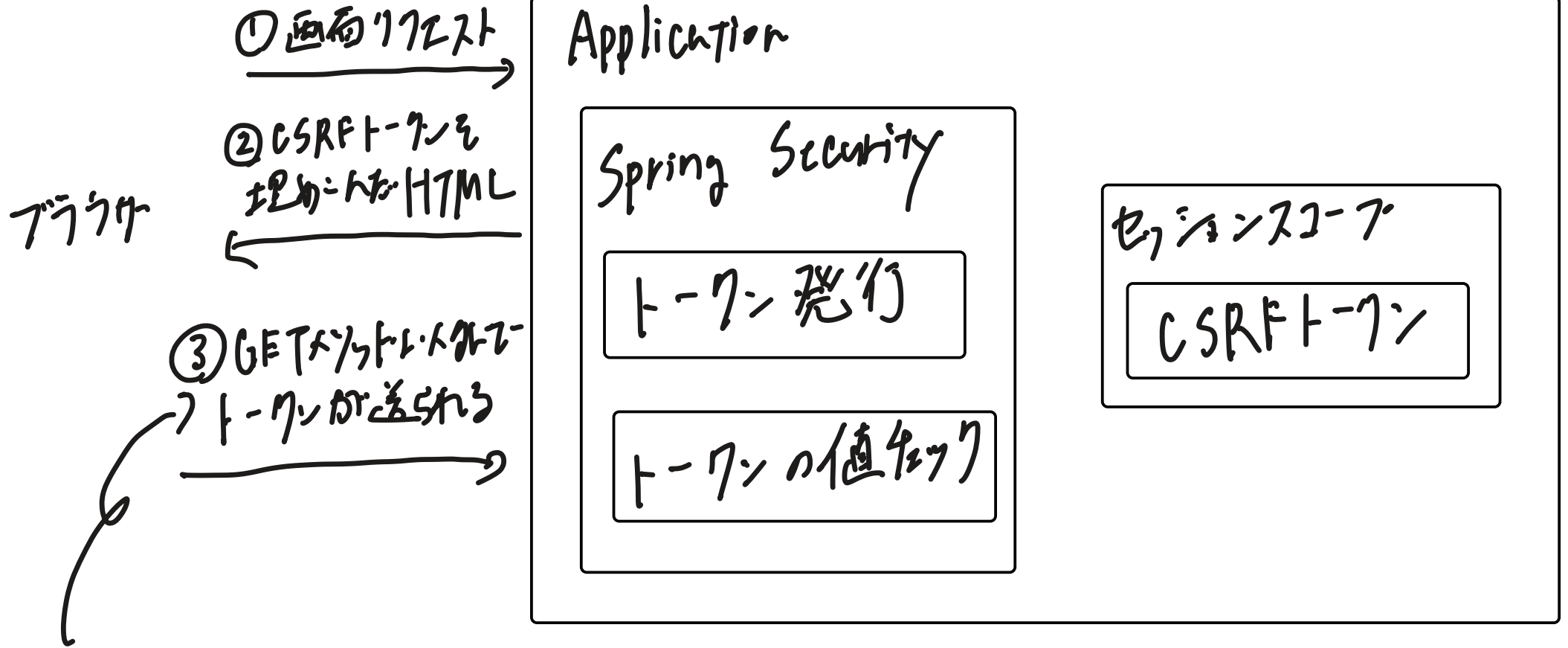


↓ ② 認可判定依頼

Authorization
Manager

CSRF対策

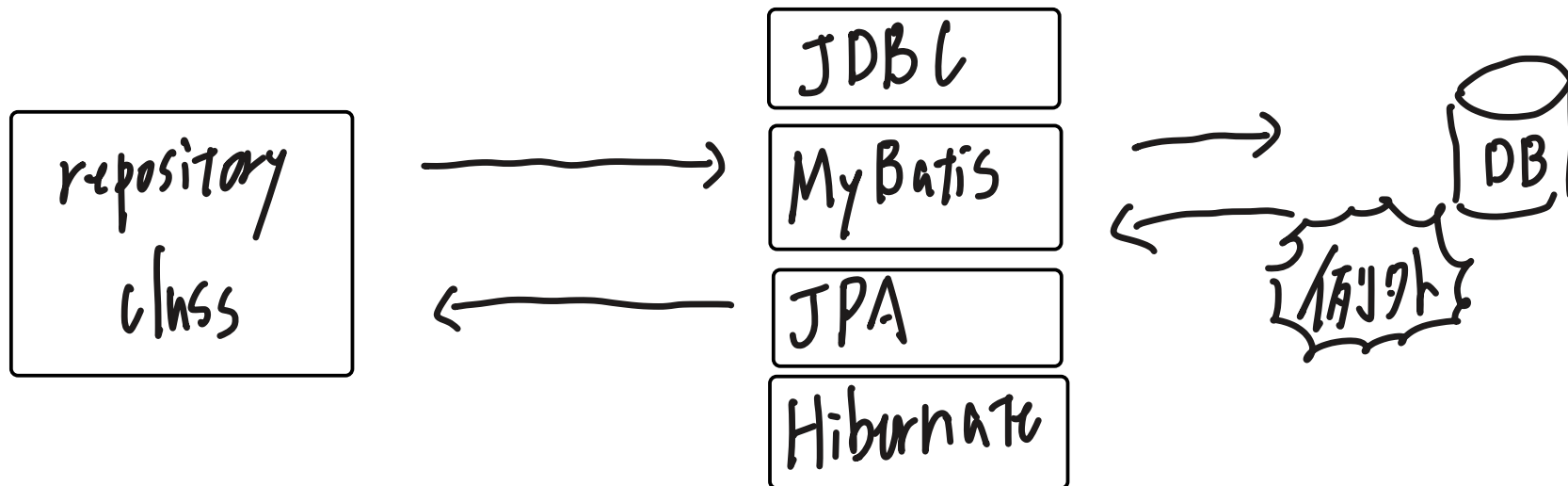
Spring SecurityのCSRFトークンの発行と値をGETする



リクエストするFormは

th: action属性を利用する必要がある

- ・ データベースアクセス時の例外
spring を利用しない場合



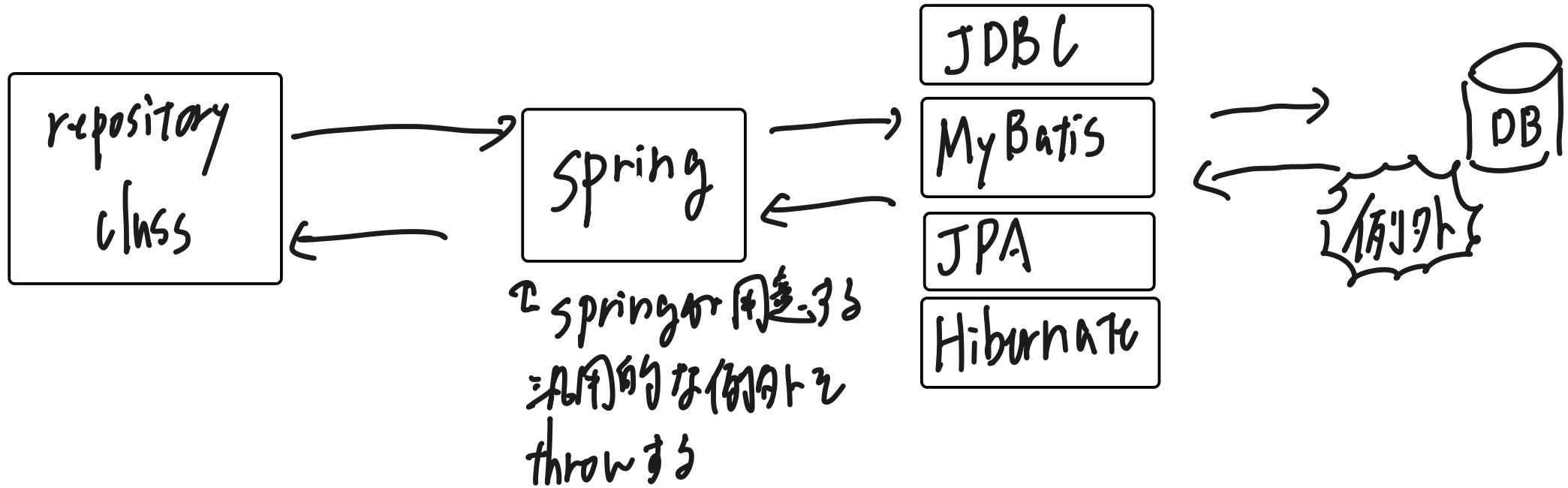
利用されるライブラリによって
throw される例外が変わる

=> catch する例外が変わる

=> ライブラリに依存したコードになる

データベースアクセス時の例外

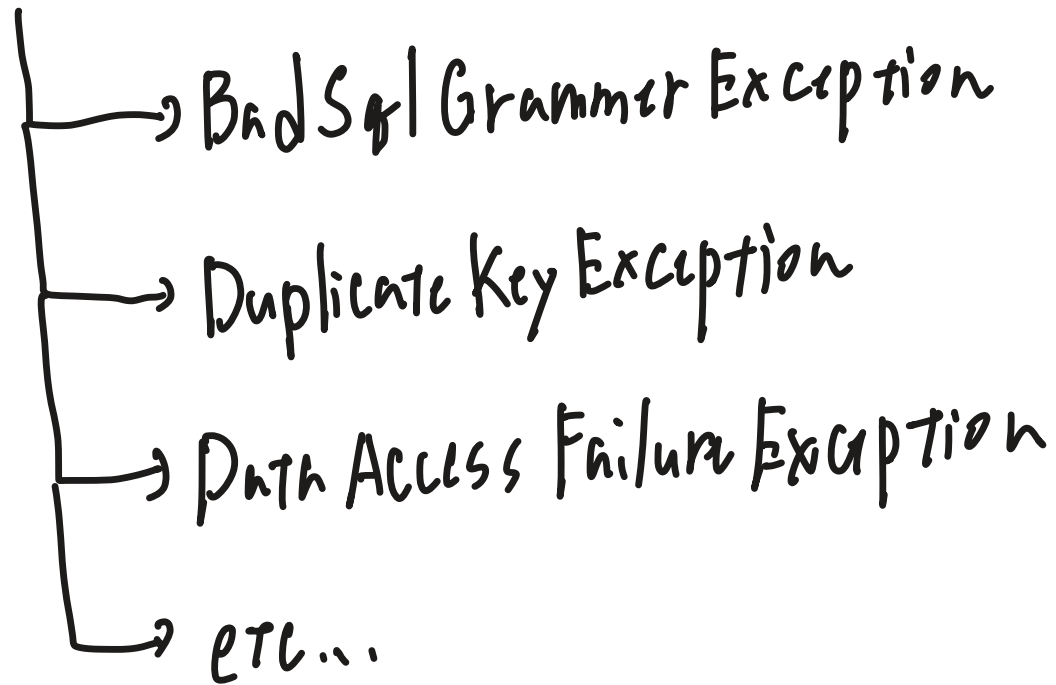
- Spring を利用する場合



⇒ ライブラリに依存しないコードになる

⇒ 利用するライブラリを変更するのが
容易になる

- SpringがthrowするDB系のException
 - Data Access Exception



※ 全て非デフォルト例外 = 発生する位置を知られる

Transaction の伝播

Foo Service
@Transactional
foo Method()
 barservice.barMethod



Bar Service
@Transactional
bar Method

Transaction の伝播

Transaction

⦿
(上記処理の場合)

or

Transaction

+

Transaction

⦿ 各処理毎に
Transaction のコミットを行う
⇒ データ不整合が
起こる可能性がある

Transactionの伝播の特徴

- ・ 全ての処理が終わったタイミングでコミットする
⇒ 一つでも処理が失敗すると全てロールバックする



Transactionの伝播を同一にする場合

- ・ 各処理でTransactionを取得し、
各処理が終わったタイミングでコミットする
⇒ 各処理毎にデータを登録出来る。
ただし、データ不整合になる可能性がある。

Transactionの伝播を利用するのびじり

@Transactionalのpropagation属性の値で切り替える

- ・ REQUIRED: 伝播する (デフォルト)
- ・ REQUIRED_NEW: 伝播しない

Transactionの伝播を利用しない時の実装の注意点

↳ 同一クラス内の別メソッドを呼び出す場合は、

propagation属性に REQUIRED-NEWを指定しても必ず伝播する

Foo Class

@Transactional

foo()

def();

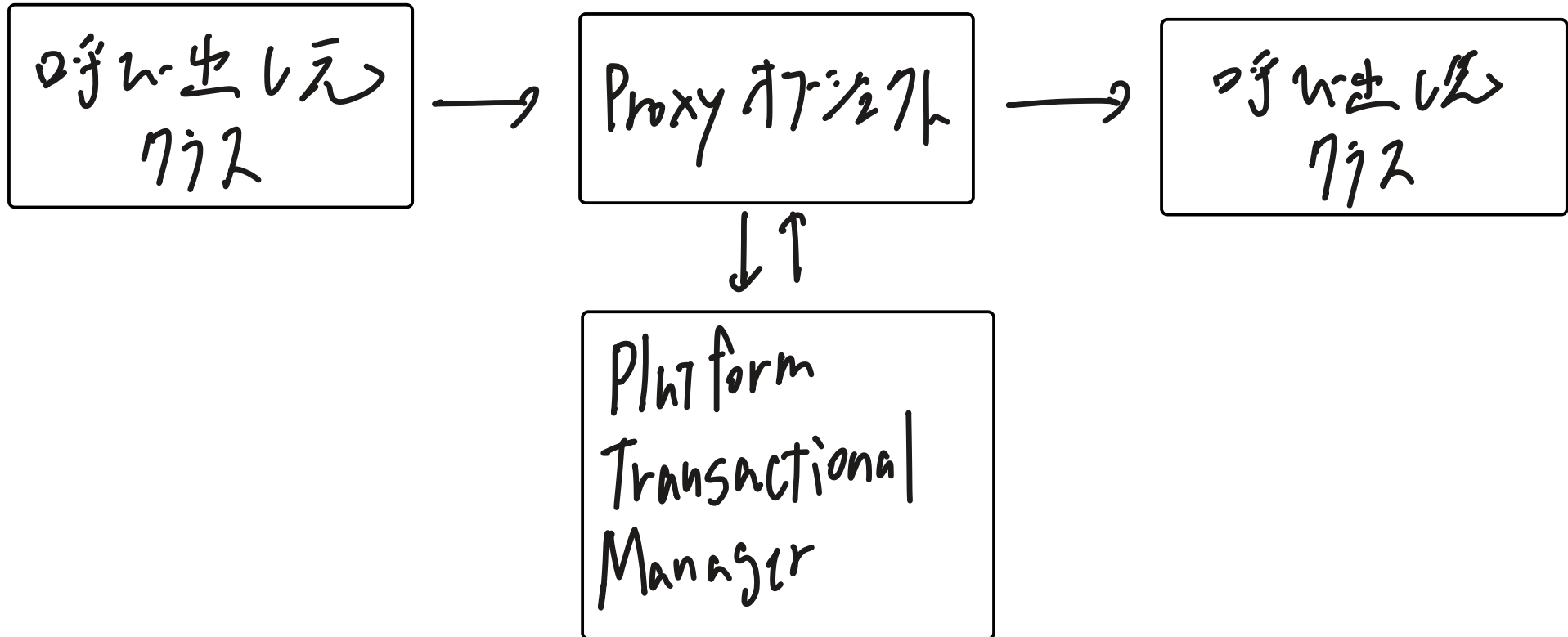
@Transactional(propagation=REQUIRED-NEW)
def()

Transactionの伝搬が必ず発生する理由

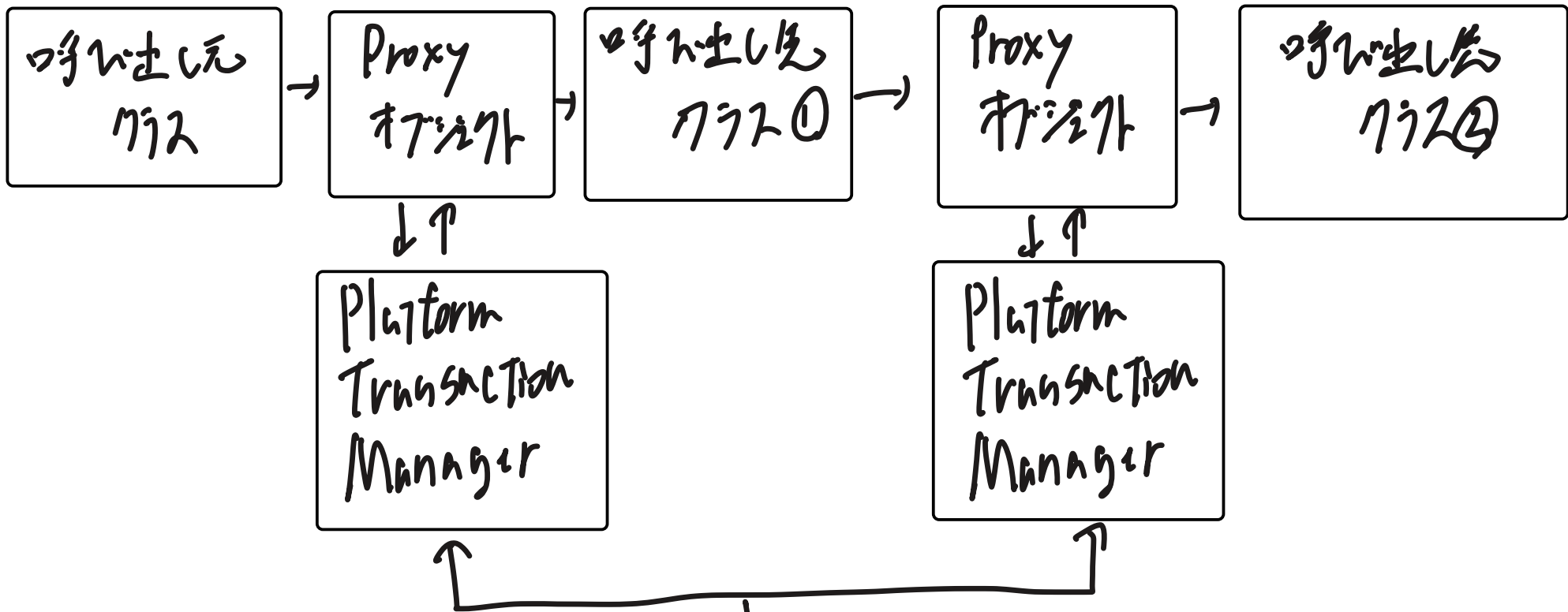
Transactionの管理は @Transactional が定義されているクラスの

Proxy オブジェクト (正しくは Platform Transactional Manager) が行う。という

⇒ 同一クラスの場合、1つの Proxy オブジェクトが Transactionの管理を行うため、必ず伝搬する



伝搬しない時



各クラスのTransactionを管理する
=>伝搬している、新しいTransactionを作る
必要がある

・セッションスコープ

⇒ブラウザが毎に保存したデータを保存出来る領域を
(X)セッションカートなど

・セッションスコープのBean

・ @Session Scope を付加する

↳ 上記のステレオタイプアノテーションを付加することで

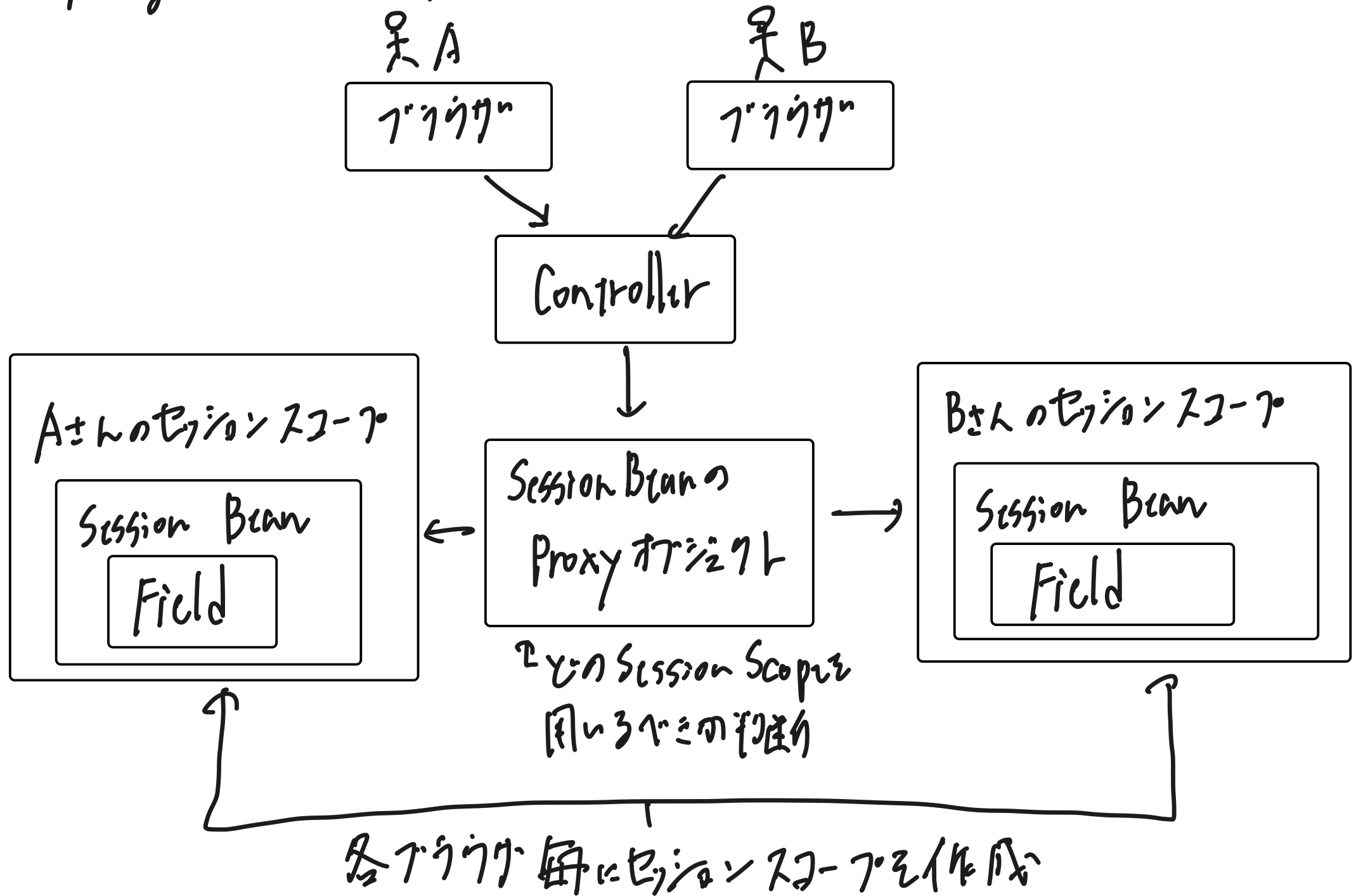
Bean対象であること (Component Scanの対象であること) を明示する

・ implements Serializable をつける

※セッションスコープ: 保存するにはシリアライゼーションが必要

※ @SuppressWarnings("serial") をつける

Spring の Session Scope Bean

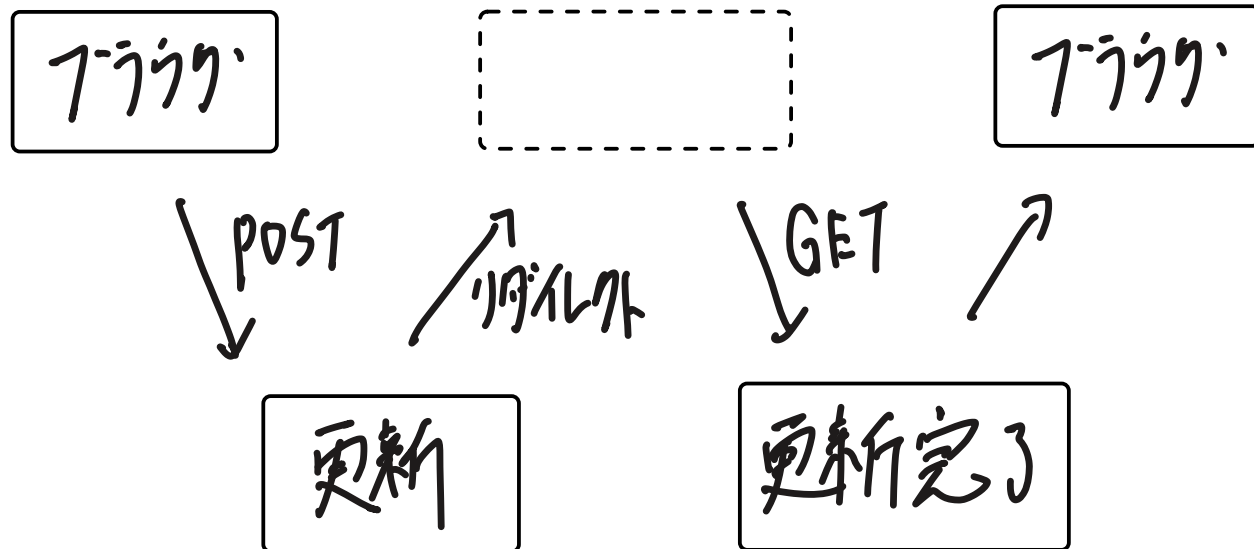


Flash Scope

・リダイレクト元からリダイレクト先にデータを運搬するための領域

・リダイレクトの利用シーン
更新処理の場合

PRGパターン



更新完了画面を

リポートしても更新完了画面が
再表示されるだけ。

更新処理は呼び出されず
データ整合性(多重リクエスト)が
発生しない

・リダイレクトの問題点

データ連携方法

- ・ Model: リクエストスコープのため、リダイレクト先へ連携出来ない

- ・ Session: リダイレクト先で利用可能

ただし、Sessionは明示的に破棄しないと

メモリに貯まり続けるため、適切なタイミングで

破棄が必要

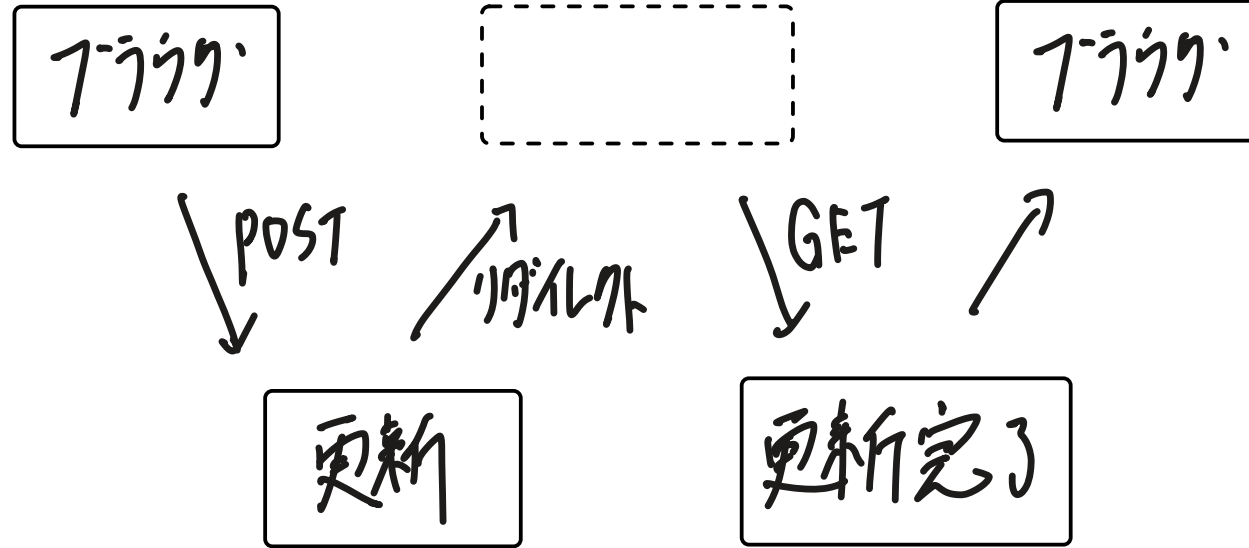
※ リダイレクト先表示時に破棄するのは

破棄して良いタイミングから遠いので非推奨

★ フラッシュスコープ: リダイレクト先へ連携可能で

リダイレクト先表示時にデータを破棄が可能

🔗 スコア



リクエスト スコア

アップロード スコア

セッション スコア

Spring Security の認証・認可したユーザー情報

↳ Security Context に格納

- ・ 格納内タイミンク : Security Filter で認証が出来る時
- ・ 格納先 : セッションスコープ ... 認証済みのユーザを確認する場所
Thread Local ... 実行処理が必要時の取得場所

プロパティの定義場所

- ・ OS の環境変数
- ・ システムプロパティ (java コントロール指定)
- ・ プロパティファイル (.properties)

プロパティの読み込みと格納先

- ・ 環境変数
- ・ システムプロパティ
- ・ プロパティファイル

} DIコンテナ生成時
自動的に読みこまれる

} spring boot の
オートコンフィグレーション

格納先: Environment オブジェクト

- ・ 独自のプロパティファイル (application-プロパティファイル) を
読み込む方法

- ・ Java Config に @PropertySource (プロパティファイル) を付ける

- ・ Java Config の @Profile をつけることで切り替えも可能

- ・ Spring boot を利用する場合

起動するプロパティファイルのプロパティファイル + application.propertiesを読み込む

→ Spring 起動時の Java コードで指定

プロパティファイルを指定せずに起動した場合、プロパティ: default が適用される

プロパティファイルが複数ある場合はプロパティファイルの指定優先度

・プロパティの取得方法

@ Value: Environment オブジェクトから直接取得

@ Configuration Properties: prefix から同一のグループを

管理するリストを作成し、その中で取得する。

④ @Component などの Bean として扱う

⇒ DIコンテナ生成時に値が格納される。

インジェクションが可能

多分

{ param. 000 } は VARIABLE -

GETメソッドのバリデーションの役割 (Handler Adapter 2 - Controller 1)

⇒ Controller 1772の @Validated
メソッドがある (1772)

POSTメソッドのバリデーションの役割

⇒ Handler Adapter